16-tap Gaussian Filter for Neural Data Smoothing (RTL → PnR)

Saima
B.Tech Project

Guide: Prof. Laxmeesha Somappa
Dept. of Electrical Engineering

Aug 2025 - Nov 2025

Outline

1. Motivation
2. Background
3. Architecture
4. Functional Blocks
5. Verification & Synthesis
6. Results
7. Future Improvements
8. Results
9. Conclusion
10. Acknowledgements

Motivation

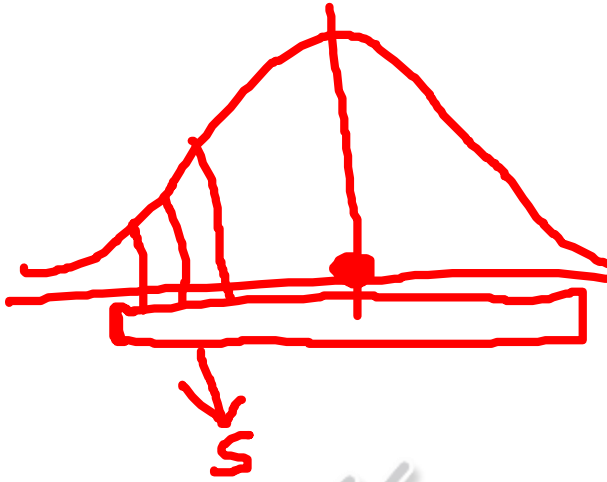
- **The Challenge:** Neural signals recorded from the brain are inherently noisy and contain high-frequency artifacts.
- **The Need:** Raw neural spikes must be smoothed into a continuous firing rate for accurate motor intention decoding.
- **The Constraint:** Implantable BMIs have strict power and silicon area budgets.
- **Goal:** Design a hardware accelerator that performs complex mathematical smoothing efficiently, minimizing silicon area without sacrificing precision.

Background: Gaussian Smoothing

- Implements a Finite Impulse Response (FIR) Filter.
- **Operation:** Convolution of input spikes with a bell-shaped kernel.
- **Equation:**

$$y[n] = \sum_{k=-7}^{8} h[k] \cdot x[n-k]$$

- **Specifications:**
  - 32 Neural Channels.
  - 16-tap Kernel (Fixed-point).
  - 4-bit Input → 10-bit Output.

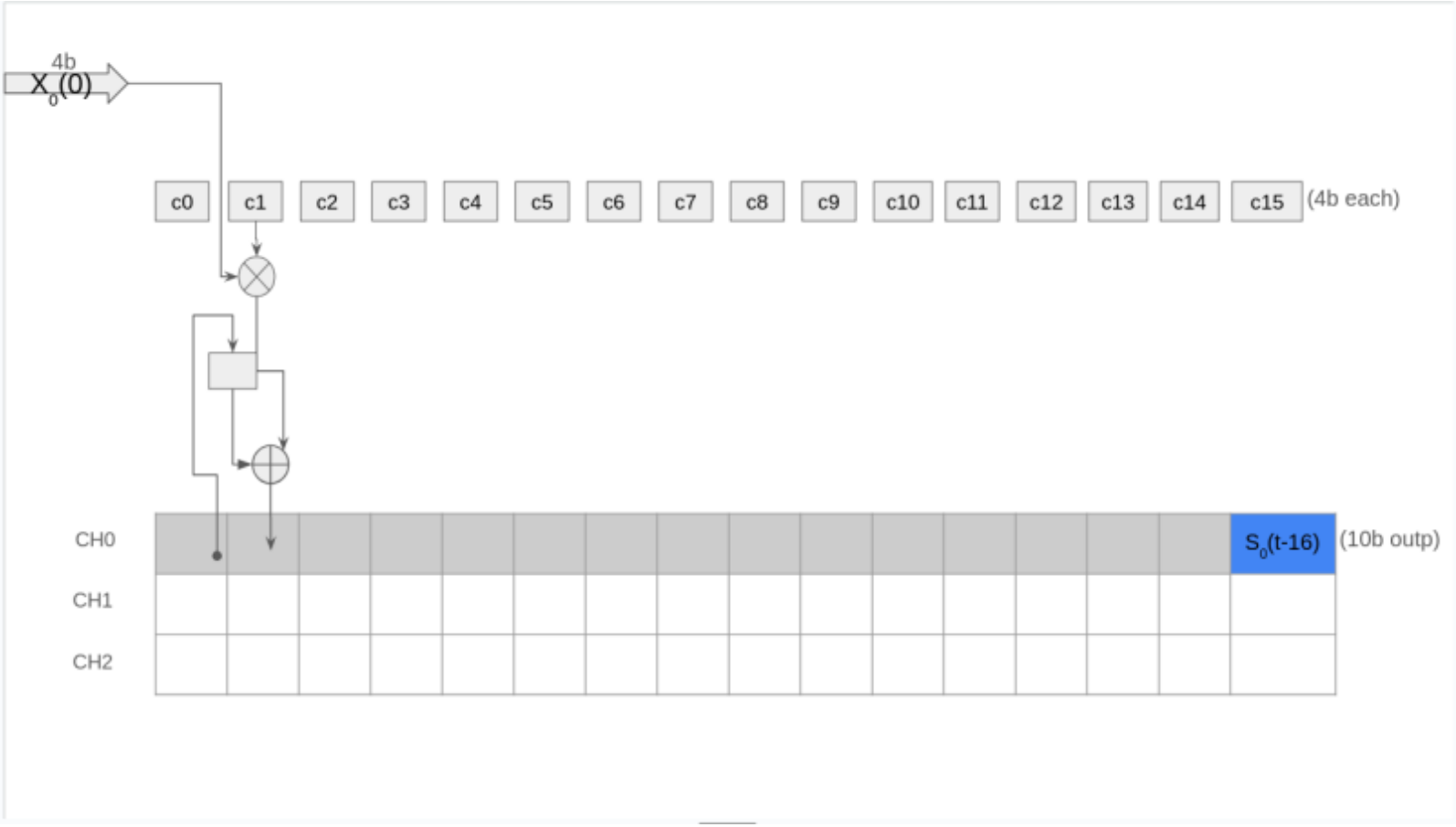


Architecture Exploration

1. **Baseline Architecture (Parallel/Spatial)**
   - Uses **16 Multipliers** and **15 Adders** simultaneously.
   - *Pro:* High Throughput (1 cycle latency).
   - *Con:* Excessive area (16 DSP Slices); wasteful for low-frequency neural signals.

2. **Proposed Architecture (Sequential/TDM)**
   - Uses **1 Multiplier** and **1 Adder**.
   - **Time Division Multiplexing (TDM):** Reuses the single MAC unit 16 times per sample.
   - *Result:* ~**94% Reduction** in arithmetic logic.

System Block Diagram



32

clk freq = 8 MHz

Clock Cycle 1

chnum = 0

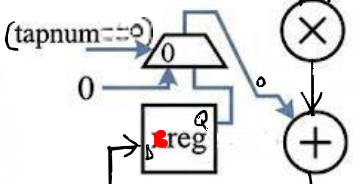
tapnum = 0

Input(4b)

Coefficient Memory (4b coeff)

$x_0(0)$

C_{-7}	C_{-6}	C_{-5}	C_{-4}	C_{-3}	C_{-2}	C_{-1}	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8



(10b) Partial Sum Memory

Ch 0	$x_0(-7)$ 0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Ch 1																	

clk freq = 8 MHz

Clock Cycle 2

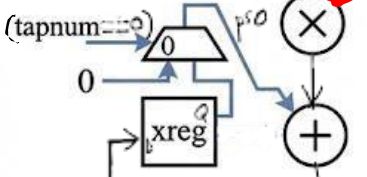
chnum = 0

tapnum = 1

Input(4b) $x_0(0)$

Coefficient Memory (4b coeff)

C_{-7}	C_{-6}	C_{-5}	C_{-4}	C_{-3}	C_{-2}	C_{-1}	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8



(10b) Partial Sum Memory

Ch 0	ps0 $x_0(-7)$	0 + $x_0(-6)$	ps1 0	ps2 0	ps3 0	0	0	0	0	0	0	0	0	0	0	0
Ch 1																

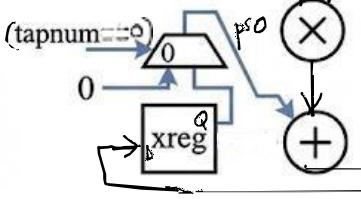
clk freq = 8 MHz

Clock Cycle 15 chnum = 0 tapnum = 15

Input(4b) $x_0(0)$

Coefficient Memory (4b coeff)

C_{-7}	C_{-6}	C_{-5}	C_{-4}	C_{-3}	C_{-2}	C_{-1}	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8



goes on till tap num == 15

(10b) Partial Sum Memory

	ps0	ps1	ps2	ps3	ps4											
Ch 0	$x_0(-7)$	0 + $x_0(-6)$	0 + $x_0(-5)$	0 + $x_0(-4)$												0 + $x_0(8)$
			0	0	0	0	0	0	0	0	0	0	0	0	0	0
Ch 1																

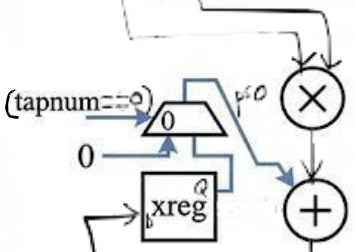
clk freq = 8 MHz

Clock Cycle 17 chnum = 1 tapnum = 0

Input(4b) $x_1(0)$

Coefficient Memory (4b coeff)

C_{-7}	C_{-6}	C_{-5}	C_{-4}	C_{-3}	C_{-2}	C_{-1}	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8



Same procedure for Ch 1 and the rest of the channels.
So, After 32x16 clock cycles, we will get the inputs for the next TIMESTAMP with Ch0, Ch1 ... Ch31 data coming in serially.

(10b) Partial Sum Memory

	ps0	ps1	ps2	ps3	ps4											
Ch 0	$x_0(-7)$	0 + $x_0(-6)$	0 + $x_0(-5)$	0 + $x_0(-4)$	-											0 + $x_0(8)$
			0	0	0	0	0	0	0	0	0	0	0	0	0	0
Ch 1	$x_1(-7)$															

clk freq = 8 MHz

chnum = 0

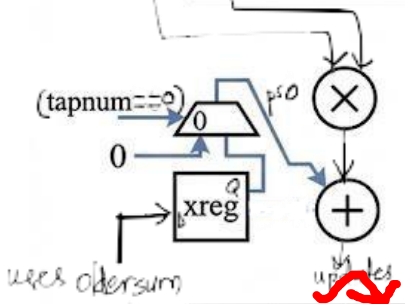
tapnum = 0-15.

timestamp=1

Input(4b) $x_0(1)$

Coefficient Memory (4b coeff)

C_{-7}	C_{-6}	C_{-5}	C_{-4}	C_{-3}	C_{-2}	C_{-1}	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8



Let's look at how the smoothened output is computed only for channel 0

We will get a new input after every 32x16 clock cycles

(10b) Partial Sum Memory

	ps0	ps1	ps2	ps3	ps4											
Ch 0	$x_1(-7)$	$x_0(-7)$ + $x_1(-6)$	0 + $x_0(-6)$ + $x_1(-5)$	0 + $x_0(-5)$ + $x_1(-4)$	0 + $x_0(-4)$ + $x_1(-3)$	$x_0(-3)$ $x_1(-2)$	$x_0(-2)$ $x_1(-1)$	$x_0(-1)$ $x_1(0)$	$x_0(0)$ $x_1(-1)$							0 + $x_0(7)$ + $x_1(8)$
Ch 1																

clk freq = 8 MHz

chnum = 0

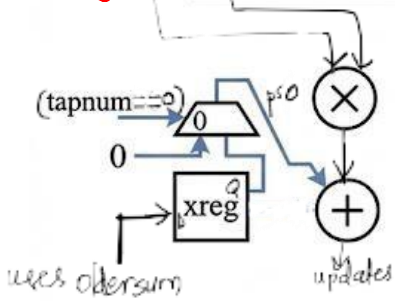
tapnum = 0 - 15 .

timestamp = 2

Input(4b) $x_2(2)$

Coefficient Memory (4b coeff)

C_{-7}	C_{-6}	C_{-5}	C_{-4}	C_{-3}	C_{-2}	C_{-1}	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8



Let's look at how the smoothened output is computed only for channel 0

We will get a new input after every 32x16 clock cycles

(10b) Partial Sum Memory

	ps0	ps1	ps2	ps3	ps4											
Ch 0	$x_2(-7)$	$x_1(-7)$ + $x_2(-6)$	$x_0(-7)$ + $x_1(-6)$ + $x_2(-5)$	0 + $x_0(-6)$ + $x_1(-5)$ + $x_2(-4)$	0 + $x_0(-5)$ + $x_1(-4)$ + $x_2(-3)$	0 + $x_0(-4)$ + $x_1(-3)$ + $x_2(-2)$	$x_0(-3)$ + $x_1(-2)$ + $x_2(-1)$	$x_0(-2)$ + $x_1(-1)$ + $x_2(0)$	$x_0(-1)$ + $x_1(0)$ + $x_2(1)$	$x_0(0)$ + $x_1(1)$ + $x_2(2)$	$x_0(1)$ + $x_1(2)$ + $x_2(3)$	x				0 + $x_0(6)$ + $x_1(7)$ + $x_2(8)$
Ch 1																

clk freq = 8 MHz

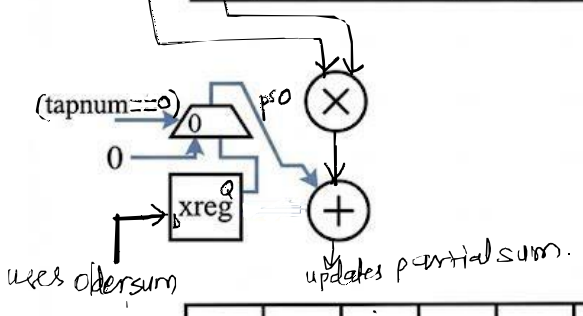
chnum = 0

tapnum = 0-15 . timestamp = 15

Input(4b) $x_0(2)$

Coefficient Memory (4b coeff)

C_{-7}	C_{-6}	C_{-5}	C_{-4}	C_{-3}	C_{-2}	C_{-1}	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8



At timestamp = 15, this is what we get in the last element of partial sum memory is

$$x_0C_{-7} + x_1C_{-6} + x_2C_{-5} + x_3C_{-4} + x_4C_{-3} + x_5C_{-2} + x_6C_{-1} + x_7C_0 + x_8C_1 + x_9C_2 + x_{10}C_3 + x_{11}C_4 + x_{12}C_5 + x_{13}C_6 + x_{14}C_7 + x_{15}C_8$$

(10b) Partial Sum Memory

Ch 0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Ch 1																

clk freq = 8 MHz

chnum = 0

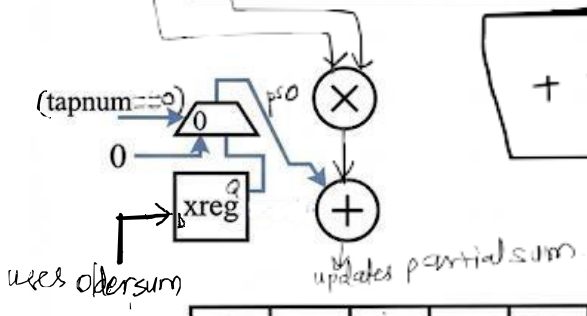
tapnum = 0-15 .

timestamp = 15

Input(4b) $x_0(2)$

Coefficient Memory (4b coeff)

C_{-7}	C_{-6}	C_{-5}	C_{-4}	C_{-3}	C_{-2}	C_{-1}	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8



$$x_0C_{-7} + x_1C_{-6} + x_2C_{-5} + x_3C_{-4} + x_4C_{-3} + x_5C_{-2} + x_6C_{-1} + x_7C_0 + x_8C_1 + x_9C_2 + x_{10}C_3 + x_{11}C_4 + x_{12}C_5 + x_{13}C_6 + x_{14}C_7 + x_{15}C_8$$

$\Rightarrow$ which is equal to $y[7]$!!!

After this, we get $y[8]$, $y[9]$, etc as we get the next timestamp's input.

Ch 0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Ch 1																

Functional Blocks

Datapath

- **MAC Unit:** A single DSP slice performing $P = (A \times B) + C$.
- **Coefficient ROM:** Synthesizable Look-Up Table (LUT) storing the 16-tap Gaussian kernel.
- **Accumulator Memory:** 32-word register file storing partial sums for each channel.

Control Logic

- **Tap Counter (4-bit):** Iterates through kernel weights (0–15).
- **Channel Counter (5-bit):** Manages channel switching (0–31).
- **State Logic:** Handles Reset, Input Latching, and Output Registration.

Verification & Synthesis Flow

- **Co-Simulation Environment:**
  - **Python (SciPy/NumPy):** Generated "Golden Vectors" (ideal floating-point outputs) and quantized test data.
  - **Vivado (Verilog):** Testbenches using `$readmemh` and `$fstrobe`.
- **Key Challenges Solved:**
  - Resolved simulation race conditions between testbench logging and registered outputs.
  - Validated 2-cycle latency in the sequential pipeline.
- **Synthesis (Cadence Genus) and PnR (Cadence Innovus):**
  - Evaluated Area-Power-Timing (PPA) trade-offs.

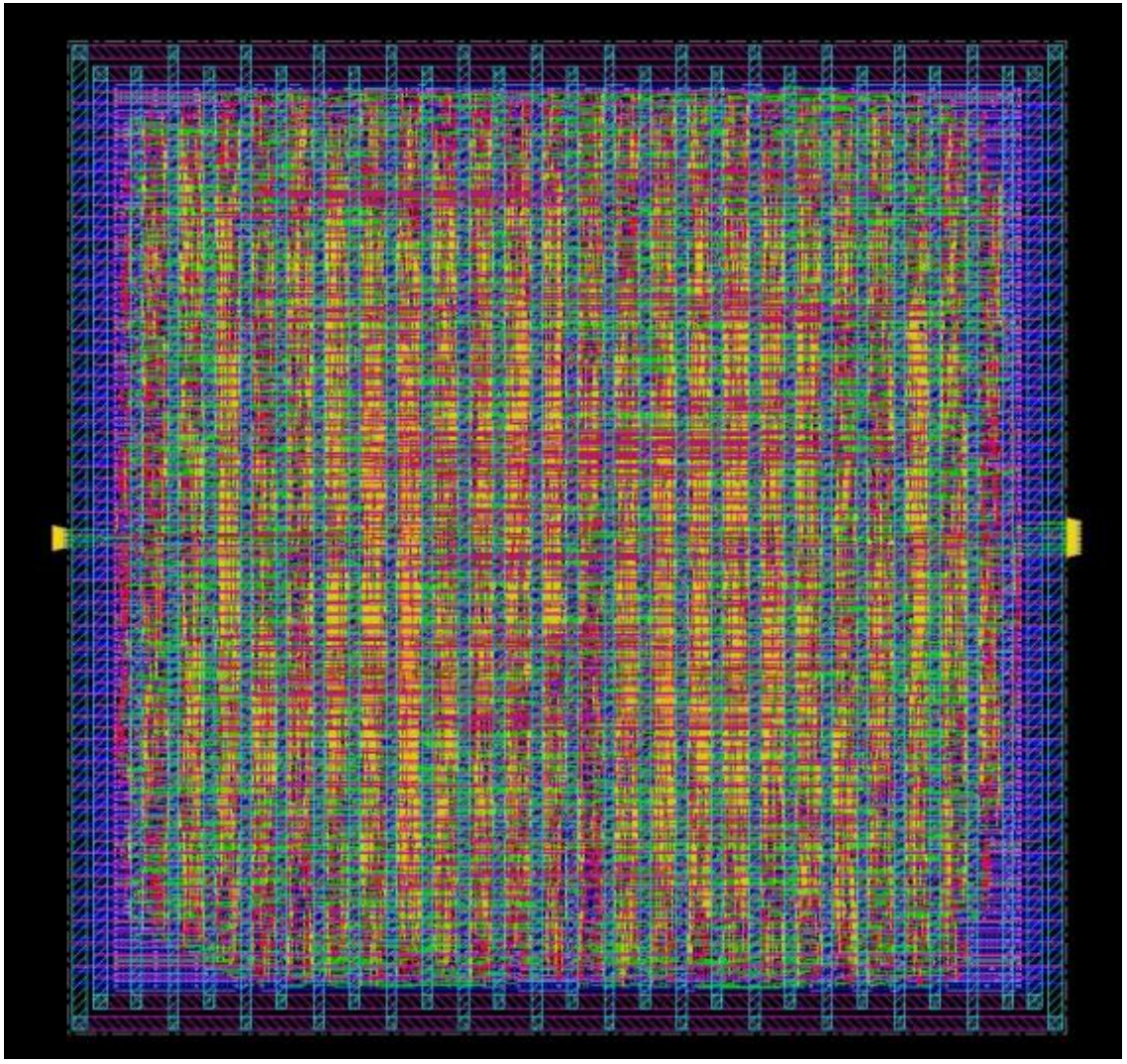
Synthesis Results (Genus)

Metric	Value
Total Area	81485.535 μm^2
Cell Count	9049
Worst Slack	120.262 ns
Power	0.841495 mW μW

Post-Route Results

Metric	Value
Setup Slack (WNS)	+117.639 ns
Hold Slack (WHS)	0.000 ns
Total Area (post-route)	59168.520 μm^2
Total Power	266 μW
Physical Verification	DRC / LVS / Antenna: Clean

Final Layout



Future Improvements

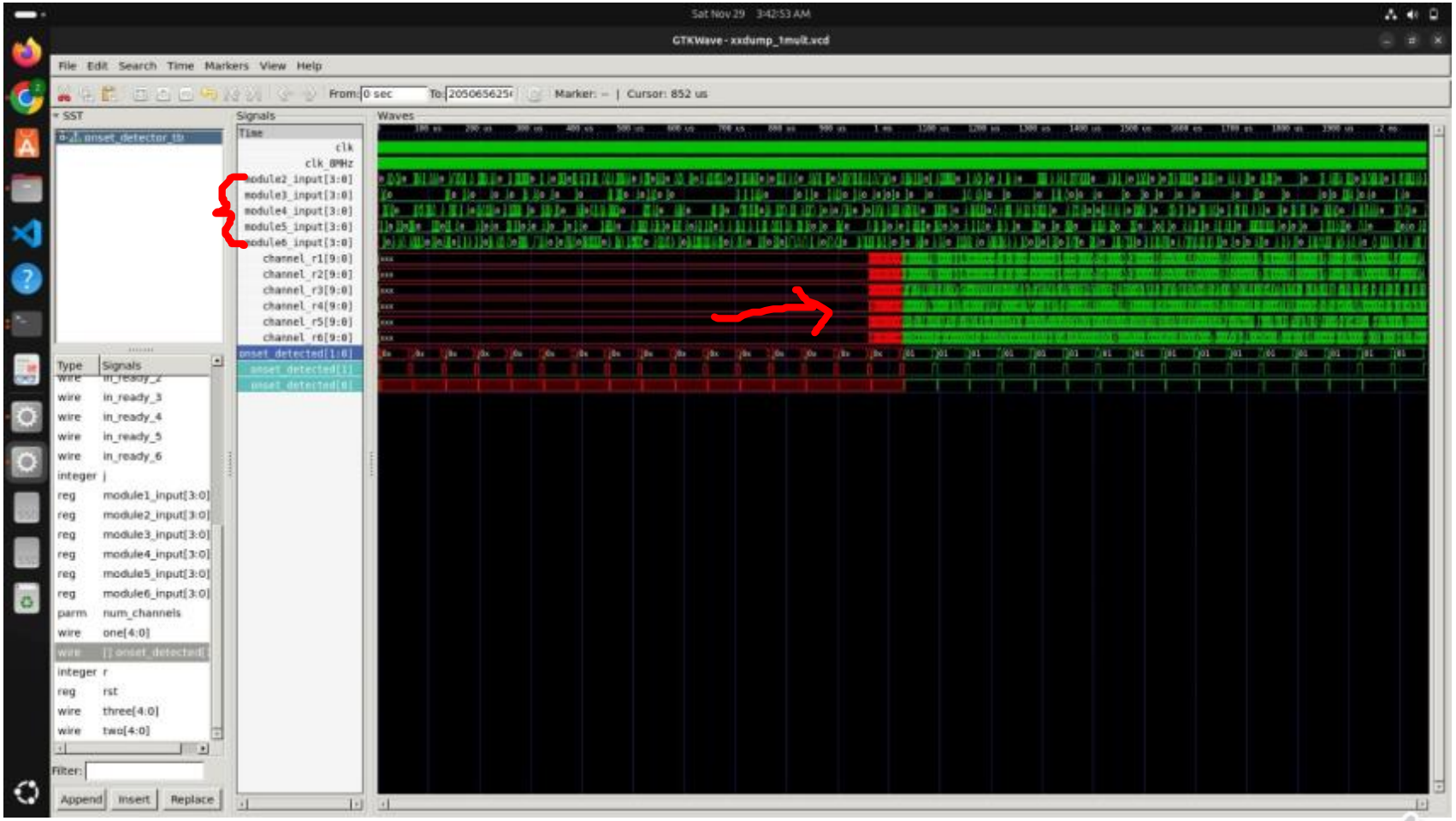
- The current design uses 31 parallel MAC units (one per class) for 32-class accumulation.
- Area can be reduced significantly by replacing them with **8 shared MAC units**.
- These 8 MAC units can be time-multiplexed (serialized) across all 32 classes.
- A **higher operating clock frequency** can be used to maintain timing performance despite serialization.
- This approach keeps throughput intact while reducing silicon area and power.

Results Summary

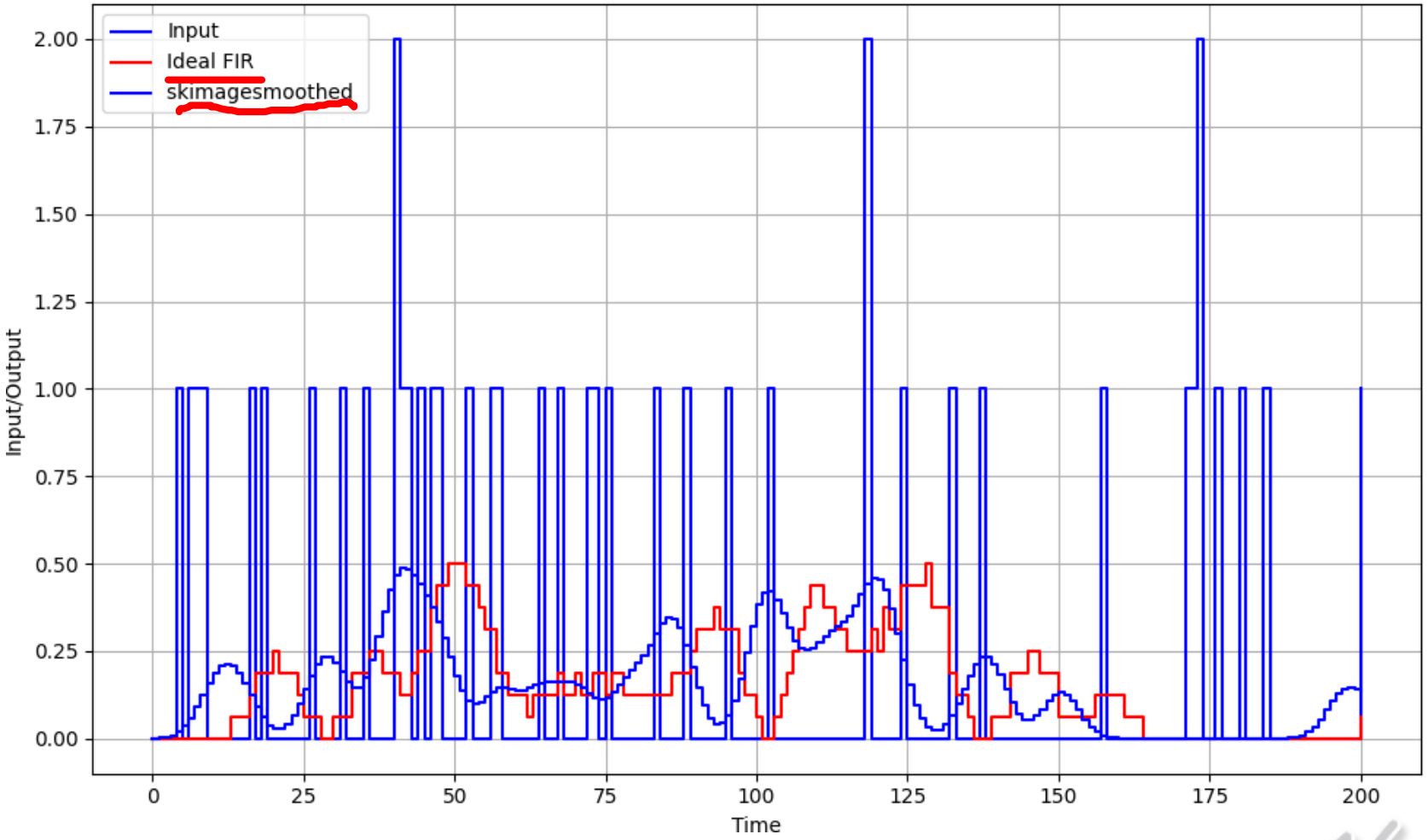
Table: Performance & Utilization

Metric	**Parallel Design**	**Proposed (TDM)**
Multipliers	16	**1**
Adders	15	**1**
Arithmetic Reduction	-	**~94%**
Latency	1 Cycle 512kHz	16 Cycles 8 MHz
Accuracy	100%	**100% (Bit-true)**

Simulation Results



Simulation Results



Conclusion

- Successfully designed and implemented a **32-channel Gaussian Smoother** in Verilog.
- Demonstrated that **Time Division Multiplexing** is a viable strategy for neural signal processing, offering massive area savings.
- Validated the design through a robust **Python-to-Verilog verification flow** and industry-standard synthesis tools.

Acknowledgements

- I express my sincere gratitude to my guide, **Prof. Laxmeesha Somappa**, for his continuous guidance on digital architecture and design principles.
- I am grateful to PhD mentors Deeksha and Ankur, for their continuous support, technical guidance, and motivation throughout the development of this work.
- I also extend my heartfelt thanks to my classmates, Shrutika Mohalkar and Kanak Dudi, for their support, insightful discussions, and collaboration over the course of this project.
- A more detailed report, along with vcd files, can be found on this link.